

Estructuras de Datos y Algoritmos

Tema 4: Arreglos

Dr. Jorge Armando Ramírez Díaz

Universidad Nacional Autónoma de México
ENES Unidad Juriquilla
Licenciatura en Matemáticas para el Desarrollo

Definición: arreglo unidimensional

Colección de n elementos, identificados por su posición:

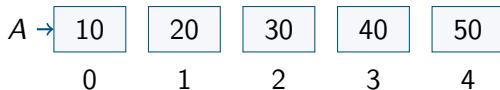
$$A = (a_0, a_1, \dots, a_{n-1}), \quad A[i] = a_i, \quad 0 \leq i < n.$$

Su operación fundamental es el **acceso directo por índice**.

- El índice indica una posición, no el valor almacenado.
- La posición importa; los valores pueden repetirse.
- Acceder a $A[i]$ no requiere recorrer los elementos anteriores.

Es una estructura **lineal**; sus elementos no tienen que estar ordenados de menor a mayor.

Una forma de implementar un arreglo **unidimensional** es almacenar sus celdas consecutivamente en memoria: una representación **contigua**. Unidimensional describe el uso de un solo índice; contiguo describe la disposición en memoria.



Un número constante de operaciones permite localizar cualquier celda: $T_{\text{acceso}}(n) = \Theta(1)$

Índices en Python

```
A = [10, 20, 30, 40, 50]
B = (10, 20, 30, 40, 50)
```

```
A[0]      # 10
A[3]      # 40
A[-1]     # 50
len(A)    # 5
```

```
A[2] = 99  # Permitido
B[2] = 99  # TypeError
```

- Los índices comienzan en cero.
- Los negativos cuentan desde el final: `A[-1]` equivale a `A[n-1]`.
- `A[n]` produce `IndexError`.
- `[]` tiene longitud cero: no tiene primer ni último elemento.

list y **tuple** permiten indexación y consultar su longitud.

list: permite reemplazar elementos y cambiar su longitud.

tuple: no permite reemplazar elementos ni cambiar su longitud.

Operaciones con listas

Cada ejemplo parte de una lista nueva: `A = [10, 20, 30]`.

Operación	Python	Efecto o resultado
Acceder por índice	<code>A[1]</code>	20
Consultar longitud	<code>len(A)</code>	3
Agregar al final	<code>A.append(40)</code>	A: [10, 20, 30, 40]
Insertar	<code>A.insert(1, 99)</code>	A: [10, 99, 20, 30]
Extraer el último	<code>A.pop()</code>	Devuelve 30; A: [10, 20]
Extraer por índice	<code>A.pop(0)</code>	Devuelve 10; A: [20, 30]
Eliminar por índice	<code>del A[1]</code>	A: [10, 30]
Buscar un valor	<code>20 in A</code>	True
Localizar un valor	<code>A.index(20)</code>	Devuelve el primer índice: 1
Obtener un segmento	<code>A[1:]</code>	Nueva lista: [20, 30]; no modifica A

`A.index(x)` produce `ValueError` si `x` no aparece. `pop()` falla si la lista está vacía.

`append` e `insert` modifican la lista y devuelven `None`: no escribimos `A = A.append(40)`.

Operaciones con tuplas

Para la tupla $T = (10, 20, 30)$:

Operación	Python	Resultado
Acceder por índice	<code>T[1]</code>	20
Consultar longitud	<code>len(T)</code>	3
Buscar un valor	<code>20 in T</code>	True
Localizar un valor	<code>T.index(20)</code>	Primer índice: 1
Obtener un segmento	<code>T[1:]</code>	(20, 30)
Concatenar	<code>T + (40,)</code>	(10, 20, 30, 40)

Estas operaciones no modifican T. Las tuplas no permiten asignar a `T[i]` ni tienen métodos como `append`, `insert` o `pop`.

`(40,)` es una tupla de un elemento: la coma es necesaria. `(40)` es simplemente el entero 40.

Costos en una lista

Operación	Python	Costo	¿Qué trabajo realiza?
Acceso / modificación	<code>A[i], A[i] = x</code>	$\Theta(1)$	Localiza la celda sin recorrer las anteriores.
Longitud	<code>len(A)</code>	$\Theta(1)$	Consulta la longitud almacenada.
Agregar / extraer al final	<code>A.append(x), A.pop()</code>	$O(1)$ amortizado	No desplaza el resto; a veces cambia la capacidad.
Insertar / eliminar	<code>A.insert(i, x)</code> <code>A.pop(i), del A[i]</code>	$O(n)$, peor caso	Desplaza las referencias posteriores; al inicio cuesta $\Theta(n)$.
Buscar / localizar	<code>x in A, A.index(x)</code>	$\Theta(n)$, peor caso	Compara hasta n elementos.
Copiar	<code>A[i:j], A.copy()</code>	$\Theta(k)$, $k \geq 1$	Crea otra lista y copia k referencias.
Concatenar	<code>A + B</code>	$\Theta(n+m)$, $n+m \geq 1$	Copia las referencias de ambas listas a otra.

Amortizado: una secuencia de operaciones al final, desde una lista vacía, tiene costo total lineal en su cantidad. Una llamada aislada puede costar $O(n)$ al cambiar la capacidad.

Suponemos índices válidos y comparaciones de costo constante. **Una llamada a un método no equivale necesariamente a una operación RAM.**

Asignar no es copiar

```
A = [1, 2, 3, 4]
B = A
B[0] = 99
print(A)
# [99, 2, 3, 4]
```

A y B se refieren a la misma lista: **aliasing**.

A.copy() o A[:] copia toda la lista. Es una **copia superficial**: si hay listas internas, estas siguen compartidas. Un corte de lista no es una operación gratuita.

Un corte crea otra lista

```
A = [1, 2, 3, 4]
B = A[1:3]
B[0] = 99
print(A)
# [1, 2, 3, 4]
```

A[i:j] incluye i y excluye j. Aquí copia dos referencias.

NumPy es una biblioteca de Python para el **cálculo numérico** y la **computación científica**.

- **Funciones matemáticas:** seno, coseno, tangente, exponenciales y logaritmos.
- **Álgebra lineal:** productos matriciales, normas y resolución numérica de sistemas de ecuaciones lineales.
- **Estadística:** medias, varianzas y desviaciones estándar.
- **Probabilidad y simulación:** generación de muestras de distribuciones como la uniforme, la normal y la binomial.

Estas herramientas permiten explorar modelos, analizar datos y simular problemas aplicados. un conjunto set no entra dentro de arreglos en python?

Definición: ndarray

Arreglo multidimensional de NumPy. Su forma, `shape`, indica el tamaño de cada eje; `dtype` indica el tipo de almacenamiento común de sus elementos.

```
import numpy as np

a = np.array([2, 4, 6],
              dtype=np.int8)
z = np.zeros(3, dtype=np.int8)
b = np.arange(0, 6, 2)
# z: [0 0 0]; b: [0 2 4]
```

- `a[0]` accede a un elemento; `a[0] = 9` lo modifica.
- `dtype=np.int64` usa enteros de 64 bits, con rango finito.
- NumPy se instala aparte; en terminal:
`python -m pip install numpy`.

Otra opción es `array.array`, incluida en Python: una secuencia de valores de un tipo básico. Aquí usaremos NumPy por sus operaciones matemáticas.

Matrices en NumPy

```
import numpy as np

M = np.array([[1, 2, 3],
              [4, 5, 6]])

M.ndim      # 2
M.shape     # (2, 3)
M.size      # 6
len(M)      # 2
```

$$M = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

```
M[1, 2]      # 6
M[0, :]      # [1 2 3]
M[:, 1]      # [2 5]
```

: selecciona todas las posiciones de ese eje.

ndim: número de ejes; shape: tamaños de los ejes; size: número total de elementos. len(M) solo cuenta el primer eje, no todas las entradas.

Operaciones numéricas

Con listas

```
A = [1, 2, 3]
B = [4, 5, 6]

A + B
# [1, 2, 3, 4, 5, 6]

2 * A
# [1, 2, 3, 1, 2, 3]
```

+ concatena y * con un entero repite.

En matrices, * multiplica elemento a elemento y @ realiza el producto matricial. Para $(r \times s)@(s \times t)$, el resultado tiene tamaño $r \times t$.

Operar sobre todo el arreglo (**vectorizar**) puede mejorar el tiempo real; $2 * a$ sigue procesando sus n elementos en tiempo $\Theta(n)$.

Con arreglos de NumPy

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b      # [5 7 9]
2 * a      # [2 4 6]
a * b      # [4 10 18]
a @ b      # 32
```

Para vectores, @ calcula el producto punto:
 $1(4) + 2(5) + 3(6) = 32$.